

MCP나 Skill 같은거 개념으로만 대충 알고 있는데 이런것들이 실제 언제 필요하고 어떻게 사용하는지 내가 진행중인 프로젝트 들을 기반으로 적용할 수 있는지 검토해서 시나리오를 알려 줘

MCP와 Skill은 역할이 다릅니다. **MCP**는 Claude가 외부 시스템(DB, API, 파일, 서비스)에 실시간으로 연결해서 데이터를 가져오거나 액션을 수행하게 해주는 "연결 통로"이고, **Skill**은 특정 작업을 일관되게 잘 수행하기 위한 "노하우 문서 (SKILL.md)"입니다. 진행 중이신 프로젝트에 대입하면 이렇게 적용하실 수 있습니다.

GD RDM 프로젝트

- MCP: 회사 보안 제약상 실제 데이터는 반출 불가하니, 스키마/메타데이터만 노출하는 로컬 MCP 서버를 하나 만드시면 좋습니다. 예:
`get_schema(column_name)` 같은 툴만 열어두고, 실제 CSV 원본 접근은 막는 방식. Schema-First 원칙을 코드 레벨로 강제하는 효과가 있습니다.
- Skill: 이미 구상 중이신 `schema-validator`, `csv-merger`, `anomaly-scanner` 서버에이전트마다 SKILL.md를 하나씩 만드시면, "이 프로젝트에서 CSV 병합할 때는 IMEINO 기준으로, 결과코드는 C/P/F만 허용" 같은 규칙을 매번 프롬프트에 안 써도 서버에이전트가 자동으로 참조합니다. 지금 겪고 계신 "매번 설명 반복" 문제를 Hook보다 더 근본적으로 줄여줍니다.

KOSPI 모니터링 시스템

- MCP: KIS OpenAPI를 로컬 MCP 서버로 감싸서 `get_price`, `get_orderbook` 같은 툴로 노출하면, Claude Agent SDK 쪽에서 "맥락적 판단"이 필요할 때만 이 MCP를 호출하는 구조가 깔끔합니다. 카카오톡 알림도 별도 MCP 툴로 분리하면 폴링 로직(순수 Python)과 판단 로직(Claude)이 명확히 분리됩니다.
- Skill: 여기는 큰 실익은 없습니다. 반복 작업이라기보다 실시간 판단이 핵심이라서요.

개인 웹사이트 (React+GitHub Pages)

- MCP: 이미 사용 가능한 GitHub 연동을 쓰시면, git 커맨드 수동 입력 없이 이슈 생성이나 배포 상태 확인을 대화로 처리하실 수 있습니다. "지금 연결

된 도구는 없지만 검색해볼까요?"보다는, 이미 GitHub API 접근이 가능한 환경이라 바로 활용 가능합니다.

- Skill: 프로젝트 구조(Tailwind v4 설정, 라우팅 컨벤션)를 SKILL.md로 남겨두면 나중에 새 세션에서 재설명 없이 이어가실 수 있습니다.

네이버 카페 다운로더

- 여기는 MCP/Skill보다는 Claude in Chrome(브라우저 에이전트) 쪽이 Selenium을 대체할 수 있는지가 관전 포인트입니다. 다만 현재 세션에서 정확한 기능 범위는 문서를 확인해야 정확한데, 필요하시면 검색해서 알려드릴 수 있습니다.

정리하면

- MCP → "지금 이 순간 외부 데이터/액션이 필요한가?"가 기준 (KOSPI 실시간 시세, GitHub 상태 조회)
- Skill → "이 작업을 앞으로도 반복하는데 매번 규칙을 설명하기 귀찮은가?"가 기준 (GD RDM의 스키마 규칙, 서버에이전트 컨벤션)

가장 우선순위가 높아 보이는 건 GD RDM의 서버에이전트별 SKILL.md 작성입니다. 지금 겪고 계신 세션 간 컨텍스트 유실 문제를 Hook 자동화와 함께 보완하는 역할을 할 수 있어서요. 혹시 이 중 하나를 실제로 구현하는 구체적인 단계 (예: schema-validator SKILL.md 작성)로 들어가 볼까요?

Hooks를 통해서 자동화를 한다는데 이게 뭐고 어떤 개념인거야?

Claude Code Hooks 개념 정리

Hooks란? Claude Code의 라이프사이클 중 특정 시점에서 자동으로 실행되는 사용자 정의 이벤트 핸들러입니다. 쉬운 말로 하면, "이 상황이 되면 무조건 이 스크립트를 실행해라"라고 미리 정해두는 자동 트리거입니다.

핵심은 **AI(모델)의 판단에 의존하지 않는다**는 점입니다. Claude가 매번 "아, 코드 포매팅 해야지" 또는 "로그 저장해야지"라고 기억해서 실행하는 게 아니라, 조건이 충족될 때마다 매번 확실하게 실행됩니다.

구성 요소 3가지

Hook 설정은 이렇게 이루어져 있습니다.

- **Event(이벤트):** 언제 실행할지 — SessionStart, PreToolUse, PostToolUse, Notification, Stop 등
- **Matcher(매치):** 어떤 조건일 때만 실행할지 걸러주는 정규식 필터 — 예를 들어 Bash 도구 호출에만, 또는 Edit/Write 작업에만 적용
- **Action(액션):** 실제로 무엇을 할지 — 보통은 셸 명령이지만, 가벼운 Claude 모델에게 평가를 맡기는 프롬프트가 될 수도 있음

주요 이벤트 예시

- SessionStart — 작업을 시작할 때 실행
- PreToolUse — Claude가 어떤 도구든 실행하기 전에 실행 (여기서 차단/수정 가능)
- PostToolUse — 도구 실행이 완료된 후 실행
- Stop — Claude가 응답을 마쳤다고 판단할 때 실행
- 최근 버전에는 SessionEnd, UserPromptSubmit, SubagentStop 등 총 20여 개 이벤트로 확장되었습니다.

핸들러 종류 4가지

command(셸 스크립트), http(외부 API 호출), prompt(경량 모델 평가), agent(도구

를 쓰는 서브에이전트 평가) — 상황에 맞게 선택할 수 있습니다.

최상길님 프로젝트 관점에서

save_dialog.py, save_session_log.py를 구현하신 접근은 정확히 이 패턴입니다. 예를 들어 Stop 이벤트나 SessionEnd 이벤트에 스크립트를 걸어두면, 세션이 끝날 때마다 Claude가 "기억해서" 저장하는 게 아니라 **무조건 자동으로** dialog.txt, session_log.txt, CLAUDE.md가 갱신됩니다. /compact로 인한 JSONL 손상 문제도, 예를 들어 PreCompact 이벤트에 스냅샷을 남기는 hook을 걸어두면 손실 없이 대응할 수 있습니다.

⚠ 한 가지 주의점: Command 혹은 사용자의 전체 시스템 권한으로 실행되므로, 설정에 추가하기 전에 모든 훅 명령을 반드시 검토하고 테스트해야 합니다. 회사 보안 정책상 검토가 필요하실 수 있습니다.

이벤트 전체를 설명해 줘

공식 문서 기준으로 전체 Hook 이벤트를 정리해 드리겠습니다. 현재 총 28개 이벤트가 있으며, 실행 시점에 따라 그룹으로 나눠서 보시면 이해가 쉽습니다.

① 세션 단위 (세션당 1회)

이벤트	발생 시점
SessionStart	세션 시작 또는 재개(--resume, /clear, 컴팩션 후) 시
Setup	--init-only, --init, --maintenance 플래그로 명시적으로 실행할 때만 (일반 시작 시엔 발생 안 함)
SessionEnd	세션 종료 시

② 턴(대화 한 번) 단위

이벤트	발생 시점
UserPromptSubmit	사용자가 프롬프트 제출 시, Claude 처리 전
UserPromptExpansion	/명령어 같은 슬래시 커맨드가 프롬프트로 확장되기 전
Stop	Claude가 응답을 마쳤다고 판단할 때 (exit 2로 계속 작업시킬 수 있음)
StopFailure	API 오류로 턴이 강제 종료될 때

③ 도구 호출(Agentic Loop) 단위 — 매 도구 호출마다

이벤트	발생 시점
PreToolUse	도구 실행 직전 (차단/입력 수정 가능) — 최상길님이 관심 있으신 rm -rf 차단 같은 게 여기
PermissionRequest	권한 승인 다이얼로그가 뜰 때
PermissionDenied	자동 모드에서 도구 호출이 거부됐을 때

이벤트	발생 시점
PostToolUse	도구 실행 성공 후
PostToolUseFailure	도구 실행 실패 후
PostToolBatch	병렬 도구 호출 묶음이 전부 끝난 후, 다음 모델 호출 전

④ 서브에이전트 관련

이벤트	발생 시점
SubagentStart	서브에이전트 생성 시
SubagentStop	서브에이전트 완료 시
TaskCreated	태스크 생성 시
TaskCompleted	태스크 완료 표시 시
TeammateIdle	에이전트 팀 멤버가 유휴 상태로 전환되기 직전

⑤ 컨텍스트/환경 관리

이벤트	발생 시점
InstructionsLoaded	CLAUDE.md나 .claude/rules/*.md가 컨텍스트에 로드될 때
ConfigChange	세션 중 설정 파일이 변경될 때
CwdChanged	작업 디렉토리 변경 시 (예: cd 실행)
FileChanged	감시 중인 파일이 디스크에서 변경될 때
PreCompact	컨텍스트 압축(compaction) 전
PostCompact	압축 완료 후

⑥ 워크트리 관련

이벤트	발생 시점
WorktreeCreate	워크트리 생성 시 (git 기본 동작 대체)
WorktreeRemove	워크트리 제거 시

⑦ MCP / 알림 / 표시 관련

이벤트	발생 시점
Elicitation	MCP 서버가 도구 호출 중 사용자 입력을 요청할 때
ElicitationResult	사용자가 응답한 후, 서버로 전송되기 전
Notification	Claude Code가 알림을 보낼 때
MessageDisplay	어시스턴트 메시지 텍스트가 화면에 표시되는 동안 (표시용 텍스트만 바꿀 수 있음, Claude가 실제로 보는 내용은 그대로)

최상길님 프로젝트에 적용해 보면

- **세션 로그 자동 저장** (save_session_log.py) → Stop 또는 SessionEnd에 걸면 됩니다.
- **대화 저장** (save_dialog.py) → Stop 이벤트가 적합합니다 (턴이 끝날 때마다 실행).
- **/compact 손상 문제** → PreCompact에서 스냅샷을 뜨고, PostCompact에서 검증하는 조합이 가장 확실합니다.
- **CLAUDE.md 자동 갱신 확인** → InstructionsLoaded로 언제 어떤 파일이 로드됐는지 로깅할 수 있습니다.

핸들러 종류 4가지 **command**(셸 스크립트), **http**(외부 API 호출), **prompt**(경량 모델 평가), **agent**(도구를 쓰는 서브에이전트 평가)에 대해서도 아주 자세히 설명해 줘. 원지 모르겠어

네, 하나씩 아주 구체적으로 설명해 드리겠습니다. 이 4가지는 "Hook이 실행됐을 때 실제로 무엇으로 처리할 것인가"에 대한 방식 차이입니다.

1. **command** (셸 스크립트)

개념: 가장 기본적인 방식입니다. 이벤트가 발생하면, 여러분이 만든 셸 명령어나 스크립트(Python, Node.js 등)를 그냥 실행시키는 겁니다.

동작 방식:

- Claude Code가 이벤트 관련 정보(JSON)를 **stdin**으로 스크립트에 넘겨줍니다.
- 스크립트는 작업을 하고, **exit code**(종료 코드)와 **stdout/stderr**로 결과를 돌려줍니다.
 - exit 0 → "문제없음, 계속 진행해"
 - exit 2 → "차단해!" (PreToolUse라면 도구 실행을 막고, Stop이라면 Claude를 계속 작업시킴)

예시 (설정 파일 `.claude/settings.json`):

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "npx prettier --write W"$CLAUDE_TOOL_INPUT_FILE_PATHW"" }
        ]
      }
    ]
  }
}
```



```

    ]
  }
]
}
}

```

→ 파일을 쓰거나 수정할 때마다 자동으로 Prettier가 돌아서 코드 포맷팅을 해줍니다.

언제 쓰나: 가장 빠르고 확실합니다. rm -rf 같은 위험한 명령 차단, 코드 자동 포맷팅, 파일 저장/로깅처럼 **명확한 규칙**이 있는 작업에 적합합니다. 최상길님의 save_dialog.py, save_session_log.py가 바로 이 타입입니다.

주의: 사용자의 전체 시스템 권한으로 실행됩니다. 샌드박스가 없어서, 스크립트가 잘못되면 파일 삭제나 정보 유출도 가능합니다.

2. http (외부 API 호출)

개념: 로컬 스크립트 대신, 지정한 URL로 이벤트 정보를 **POST 요청**으로 보내고, 그 응답을 결과로 사용하는 방식입니다.

동작 방식:

- command 혹은 똑같은 JSON을 HTTP 요청의 body에 담아서 보냅니다.
- 응답 body가 command 혹은 stdout/JSON 출력과 동일한 형식으로 처리됩니다.

예시:

```

{
  "type": "http",
  "url": "http://localhost:8080/hooks/pre-tool-use",
  "timeout": 30,

```

```
"headers": { "Authorization": "Bearer $MY_TOKEN" },  
"allowedEnvVars": ["MY_TOKEN"]  
}
```

언제 쓰나:

- **팀 전체 정책**을 한 서버에서 중앙 관리하고 싶을 때 (여러 개발자의 Claude Code가 같은 서버에 물어보도록)
- 사내 시스템(예: 사내 감사 로그 서버, Slack/사내 메신저 알림 시스템)과 연동할 때
- GD RDM 프로젝트처럼 회사 보안 정책이 있는 환경에서, 로컬 스크립트 대신 **승인된 사내 API**를 거치게 강제하고 싶을 때 유용할 수 있습니다.

차이점: 응답이 실패하거나(4xx/5xx), 연결이 끊기거나, 타임아웃이 나도 **차단하지 않고 그냥 통과**시킵니다 (non-blocking error). 즉, 외부 서버가 다운돼도 작업 흐름 자체가 멈추지는 않습니다.

3. prompt (경량 모델 평가)

개념: 셸 명령이 아니라, 이벤트 정보를 ****가벼운 Claude 모델(기본 Haiku)****에게 프롬프트로 넘겨서 "이거 괜찮은지 판단해줘"라고 물어보는 방식입니다.

동작 방식:

- 정규식이나 exit code로는 판단하기 애매한, **맥락 이해가 필요한 상황**을 다룰 때 씁니다.
- 모델이 {"ok": true/false} 같은 구조화된 답을 돌려줍니다.

예시 상황:

- "이 커밋 메시지가 사내 컨벤션에 맞는 어조인지" 판단
- "이 파일 수정이 의도한 범위를 벗어난 건 아닌지" 자연어로 판단
- 단순 정규식 매칭으로는 안 되는, 뉘앙스가 필요한 검증

언제 쓰나: command 혹처럼 100% 결정적(deterministic)이진 않지만, 단순 규칙

보다 유연한 "판단"이 필요할 때 사용합니다. 다만 매번 모델을 호출하는 것이므로 command보다 느리고 비용이 듭니다.

4. agent (도구를 쓰는 서브에이전트 평가)

개념: prompt 방식보다 한 단계 더 강력합니다. 단순히 텍스트만 보고 판단하는 게 아니라, **실제로 Read, Grep, Glob 같은 도구를 써서 파일을 직접 확인**한 뒤 판단을 내리는 미니 에이전트를 띄우는 방식입니다.

동작 방식:

- 예를 들어 "이 코드 변경이 프로젝트의 다른 파일과 충돌하는지"를 판단하려면, 단순히 변경된 diff만 보서는 부족합니다.
- Agent 혹은 관련 파일을 직접 열어보고, 프로젝트 구조를 검색해보고 나서 최종 결정을 내립니다.

예시 상황:

- "이 스키마 변경이 GD RDM의 다른 542개 컬럼 정의와 모순되지 않는지" 같은, **관련 파일을 실제로 뒤져봐야 판단 가능한** 검증
- 서브에이전트 아키텍처(schema-validator 등)와 개념적으로 유사하되, 이걸 혹은 시스템 안에서 자동으로 트리거되는 검증용 에이전트입니다

주의: 공식 문서에서도 "실험적(experimental) 기능이며 앞으로 바뀔 수 있다"고 명시하고 있습니다. 가장 강력하지만 가장 느리고 무거운 방식입니다.

정리: 4가지를 언제 골라야 할까

타입	속도	판단 방식	적합한 경우
command	빠름	규칙 기반 (if-then)	포매팅, 로깅, 명백한 위험 명령 차단
http	보통	외부 서버 위임	팀/회사 공용 정책 중앙 관리
prompt	느림	언어적 판단	컨벤션 준수, 뉘앙스 있는 검증
agent	가장 느림	파일 탐색 후 판단	프로젝트 전체 맥락 확인이 필요한 검증

실무 조언: 대부분의 경우 90% 이상은 **command**로 충분합니다. 최상길님의 save_dialog.py, save_session_log.py도 command 타입이고, 이게 정답입니다. http/prompt/agent는 "단순 스크립트로는 판단이 안 되는 특수 상황"에서만 꺼내 쓰는 옵션이라고 생각하시면 됩니다.

2, 3, 4를 잘 이해하지 못하겠어. 좀 더 구체적인 예를 하나씩 들어서 이해가 되도록 설명해 줘

네, 각각 실제로 어떤 상황에서 어떻게 동작하는지 구체적인 시나리오로 설명해 드리겠습니다.

2. http 방식 — 구체적 예시

상황 설정: 최상길님 회사에 보안팀이 있고, "직원들이 Claude Code로 회사 밖 서버(외부 API, 개인 클라우드 등)에 접속하려는 시도가 없는지 전부 기록하고, 의심스러우면 차단하라"는 정책이 있다고 가정해 보겠습니다.

command로 하면 안 되는 이유: 각 직원 PC에 로컬 스크립트를 깔아두면, 그 스크립트를 직원이 마음대로 수정하거나 지울 수 있습니다. 회사 입장에선 통제가 안 됩니다.

http로 하면:

1. 회사 보안팀이 사내 서버에 `https://security.company.com/hooks/check-command` 같은 API를 하나 만들어 둡니다.
2. 모든 직원의 `.claude/settings.json`에 이렇게 강제로 설정합니다.

```
{  
  
  "hooks": {  
  
    "PreToolUse": [  
  
      {  
  
        "matcher": "Bash",  
  
        "hooks": [  
  
          { "type": "http", "url": "https://security.company.com/hooks/check-command" }  
  
        ]  
  
      }  
  
    ]  
  
  }  
  
}
```

```
}  
  
}
```

3. 최상길님이 Claude Code에서 어떤 Bash 명령이든 실행하려고 하면, 그 명령어 내용이 사내 보안 서버로 전송됩니다.
4. 서버가 "이 명령어에 외부 IP로 curl 시도가 있네? 차단!" 이라고 판단하면 {"decision": "block"}을 돌려주고, Claude Code는 그 명령을 막습니다.
5. 서버 쪽 로직은 보안팀만 수정 권한이 있으므로, 개별 직원이 임의로 우회할 수 없습니다.

핵심: 로직이 내 컴퓨터가 아니라 회사 서버에 있다는 게 command와의 결정적 차이입니다.

3. prompt 방식 — 구체적 예시

상황 설정: 최상길님이 GD RDM 프로젝트에서 "커밋 메시지는 반드시 한국어로, 그리고 '무엇을 왜 바꿨는지'가 드러나야 한다"는 팀 규칙을 정했다고 해보겠습니다.

command(정규식)로는 왜 안 될까:

- 정규식으로는 "한국어인지"까지는 대충 체크 가능할 수 있지만, "왜 바꿨는지 이유가 드러나 있는지"는 정규식으로 절대 판단할 수 없습니다. `git commit -m "수정함"`도 한국어이고, `git commit -m "IMEINO 컬럼 파싱 로직을 정규식 기반으로 변경 — 기존 방식이 특수문자 포함 시리얼번호를 누락시키는 문제 해결"`도 한국어이지만, 품질 차이는 "의미"를 이해해야 알 수 있습니다.

prompt로 하면:

```
{  
  
  "hooks": {  
  
    "PreToolUse": [  
  
      {
```

```

    "matcher": "Bash",

    "hooks": [

        {

            "type": "prompt",

            "prompt": "다음 git commit 명령어의 커밋 메시지를 확인해줘. 한국어로 작성되었고, 무엇을, 왜 바꿨는지가 드러나면 ok:true, 아니면 ok:false와 이유를 반환해줘: $CLAUDE_TOOL_INPUT"

        }

    ]

}

]

}

}

}

```

- git commit -m "수정함" → Haiku가 읽고 "이유가 안 드러남" 판단 → {"ok": false, "reason": "변경 이유가 불명확합니다"} → 커밋이 막히고 Claude에게 다시 쓰라고 피드백이 갑니다.
- 제대로 쓴 커밋 메시지 → {"ok": true} → 통과.

핵심: "이유가 드러나는가?" 같은 **자연어 의미 판단**은 규칙(if-then)으로 못 짜지만, 작은 모델에게 물어보면 판단이 가능합니다.

4. agent 방식 — 구체적 예시

상황 설정: GD RDM 프로젝트는 542개 컬럼 스키마가 서로 연결되어 있습니다. 최상길님이 어떤 컬럼의 데이터 타입을 바꾸려 할 때, "이 변경이 다른 곳에서 이 컬럼을 참조하는 코드나 스키마 정의와 충돌하지 않는지" 확인하고 싶다고 해보겠습니다.

prompt로는 왜 부족할까: prompt 방식은 딱 그 순간 주어진 텍스트(예: 변경하려

는 diff 한 줄)만 보고 판단합니다. 그런데 "다른 파일에서 이 컬럼을 참조하는지"는 **프로젝트 전체를 뒤져봐야** 알 수 있습니다. 텍스트 하나만 던져줘서는 판단이 불가능합니다.

agent로 하면:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "agent",
            "prompt": "지금 수정하려는 스키마 파일의 컬럼 타입 변경사항을 확인하고, 프로젝트 내 다른 파일(rdm_merge.py, rdm_analysis.py 등)에서 이 컬럼을 참조하는 곳이 있는지 Grep으로 찾아본 뒤, 타입 불일치로 문제가 생길지 판단해줘."
          }
        ]
      }
    ]
  }
}
```

1. 최상길님이 스키마 파일에서 어떤 컬럼 타입을 string에서 int로 바꾸려고 합니다.
2. Edit 도구가 실행되기 직전, 이 agent 혹은 트리거됩니다.

3. 미니 에이전트가 직접 **Grep**을 돌려서 `rdm_merge.py`, `rdm_analysis.py`, `csv-merger` 서브에이전트 코드 등에서 그 컬럼명을 검색합니다.
4. "`rdm_analysis.py` 143번 줄에서 이 컬럼을 `.str.contains()`로 문자열 처리하고 있어서, `int`로 바꾸면 에러 납니다" 라고 실제로 찾아낸 뒤 차단 결정을 내립니다.

핵심: prompt는 "눈앞에 주어진 것만 보고" 판단하지만, agent는 "직접 파일을 뒤지고 다녀서" 판단한다는 게 차이입니다. 그만큼 시간도 오래 걸리고 무겁습니다.

한눈에 비교

	무엇을 보고 판단하나	비유
command	정해진 규칙(if-then)만	체크리스트대로 확인하는 검수원
http	회사 중앙 서버의 정책	본사에 전화해서 물어보는 것
prompt	눈앞의 텍스트를 읽고 이해	문장만 읽고 판단하는 검토자
agent	눈앞의 텍스트 + 관련 자료를 직접 찾아봄	관련 서류를 다 뒤져보고 판단하는 조사관

최상길님 프로젝트 규모에서는 대부분 **command**로 해결되고, 스키마 간 상호 참조 검증 같은 복잡한 케이스에서만 **agent**를 고려해보실 만합니다. `http`는 회사 차원의 강제 정책이 있을 때, `prompt`는 그 중간 정도의 "약한 의미 판단"이 필요할 때 쓰시면 됩니다.

YAML의 문법을 상세하게 정리해 주고 가장 일반적인 예를 하나 들어서 설명해 줘

YAML 문법 정리

기본 원칙

- 들여쓰기로 계층 구조 표현 (탭 금지, 공백만 사용)
- 대소문자 구분
- 파일 확장자: .yaml 또는 .yml

주요 문법 요소

1. 키-값 쌍

name: 홍길동

age: 30

2. 문자열

plain: 그냥 문자열

quoted: "따옴표가 필요한 경우: 콜론(:) 포함시"

multiline: |

 줄바꿈을 유지하는

 여러 줄 문자열

folded: >

 줄바꿈을 공백으로

 바꾸는 문자열

3. 숫자, 불리언, null

count: 42

price: 19.99

active: true

disabled: false

nothing: null

also_nothing: ~

4. 리스트(배열)

fruits:

- 사과
- 바나나
- 오렌지

인라인 표기

colors: [red, green, blue]

5. 딕셔너리(맵) 중첩

person:

name: 홍길동

address:

city: 서울

zip: "12345"

6. 리스트 안에 객체

users:

- name: 김철수

role: admin

- name: 이영희

role: user

7. 주석

이것은 주석입니다

key: value # 인라인 주석

8. 앵커(&)와 참조(*) — 중복 방지

default: &defaults

timeout: 30

retries: 3

service_a:

<<: *defaults

port: 8080

가장 일반적인 예시: 애플리케이션 설정 파일

app:

name: MyService

version: 1.2.0

debug: false

server:

host: 0.0.0.0

port: 8080

database:

type: postgresql

host: localhost

port: 5432

credentials:

username: admin

password: "\${DB_PASSWORD}"

features:

- logging
- caching
- rate_limiting

logging:

level: INFO

outputs:

- console
- file

해석하면:

- app, server, database, logging은 최상위 키(딕셔너리)
- database.credentials처럼 들여쓰기로 중첩 표현
- features는 문자열 리스트
- \${DB_PASSWORD}처럼 환경변수를 문자열 안에 넣는 것도 흔한 패턴 (실제 치환은 YAML 자체가 아니라 이를 읽는 애플리케이션이 처리)

이 형태가 Docker Compose, Kubernetes manifest, GitHub Actions 워크플로우 등에서 가장 흔히 보는 구조입니다. GD RDM 프로젝트의 설정 파일이나 CLAUDE.md 관련 메타데이터를 YAML로 관리하실 때도 이 패턴이 그대로 적용됩니다.

md와 뭐가 다르고 각각 언제 사용하는게 좋아?

YAML과 Markdown, 무엇이 다를까요?

두 포맷은 목적 자체가 다릅니다.

구분	YAML	Markdown
목적	데이터/설정 표현	문서 작성
핵심 구조	키-값, 리스트, 계층 구조	제목, 문단, 강조, 목록 등 텍스트 서식
읽는 주체	주로 프로그램(파서)이 읽음	주로 사람이 읽음
예시 용도	설정 파일, API 명세, CI/CD 파이프라인	README, 블로그 글, 문서, 노트

각각 언제 쓰면 좋은지

YAML을 쓰는 경우

- 애플리케이션 설정값을 저장할 때 (예: config.yaml)
- Kubernetes, Docker Compose, GitHub Actions 등 도구가 정해진 스키마로 데이터를 읽어야 할 때
- 구조화된 데이터(중첩된 객체, 리스트)를 프로그램이 파싱해야 할 때

Markdown을 쓰는 경우

- 사람이 읽을 문서를 작성할 때 (README, 위키, 블로그 글)
- GitHub, Notion 등에서 서식 있는 텍스트를 빠르게 작성할 때
- 목록, 표, 코드 블록 정도의 가벼운 서식만 필요할 때

재미있는 지점

둘이 겹치는 부분도 있습니다. Markdown 문서 맨 위에 **YAML Front Matter**를 넣어서 메타데이터(제목, 날짜, 태그 등)를 표현하는 방식이 대표적입니다.

title: GD RDM 프로젝트 개요

date: 2026-07-04

tags: [rdm, schema-first]

본문 내용

여기부터는 일반 Markdown 문서입니다.

이 방식은 최상길님께서 다루셨던 CLAUDE.md나 SKILL.md 같은 파일에서도 자주 쓰이는 패턴입니다 — 파일 위쪽엔 YAML로 메타데이터를 정의하고, 아래는 Markdown으로 실제 설명을 서술하는 구조죠.

한 줄 요약: YAML은 "기계가 읽을 데이터", Markdown은 "사람이 읽을 문서"라고 생각하시면 구분이 쉽습니다.

CI/CD가 뭐야?

CI/CD는 소프트웨어 개발에서 코드 변경사항을 자동으로 빌드, 테스트, 배포하는 프로세스를 말씀드립니다.

CI (Continuous Integration, 지속적 통합)

- 개발자들이 코드를 자주(하루에도 여러 번) 공유 저장소에 병합(merge)하는 방식입니다.
- 코드가 병합될 때마다 자동으로 빌드하고 테스트를 실행해서, 문제를 최대한 빨리 발견합니다.
- 예: 여러 명이 각자 기능을 개발하다가 코드를 합칠 때 충돌이나 버그가 생기는 것을 조기에 잡아냄

CD는 두 가지 의미로 쓰입니다

- **Continuous Delivery (지속적 전달):** 테스트를 통과한 코드를 언제든지 배포할 수 있는 상태로 자동 준비해두는 것 (실제 배포는 사람이 승인)
- **Continuous Deployment (지속적 배포):** 테스트를 통과하면 사람의 개입 없이 자동으로 운영 환경까지 배포하는 것

전체 흐름 예시

1. 개발자가 코드 작성 후 Git에 push
2. 자동으로 빌드 및 테스트 실행 (CI)
3. 테스트 통과 시 스테이징 환경에 배포
4. 승인 또는 자동으로 운영 환경에 배포 (CD)

GD RDM 프로젝트처럼 Git으로 버전 관리를 하고 계신 환경에서는, 이후 GitHub Actions나 GitLab CI 같은 도구로 CI/CD 파이프라인을 구축하시면 코드 변경 시 자동 테스트나 배포까지 연결하실 수 있습니다.

GitHub Actions와 GitLab CI에 대해서 자세히 알려 줘

GitHub Actions와 GitLab CI는 대표적인 CI/CD 자동화 도구입니다. 각각의 특징과 차이점을 자세히 설명드리겠습니다.

GitHub Actions

개요

- GitHub 저장소에 내장된 CI/CD 도구로, 별도 설치 없이 GitHub만 있으면 바로 사용 가능합니다.
- 저장소 내 `.github/workflows/` 폴더에 YAML 파일로 워크플로우를 정의합니다.

핵심 개념

- **Workflow:** 하나 이상의 Job으로 구성된 자동화 프로세스 (YAML 파일 하나가 하나의 워크플로우)
- **Job:** 워크플로우 내에서 실행되는 작업 단위. 여러 Job은 병렬 또는 순차 실행 가능
- **Step:** Job 안에서 순서대로 실행되는 개별 명령어나 액션
- **Action:** 재사용 가능한 작업 단위 (예: actions/checkout, actions/setup-node)
- **Runner:** 실제로 워크플로우가 실행되는 서버 (GitHub 제공 또는 Self-hosted)

예시 YAML

```
name: CI Pipeline
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  pull_request:
```

```
    branches: [main]
```

jobs:

build-and-test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4

with:

node-version: '20'

- run: npm install
- run: npm test

장점

- GitHub과 완벽하게 통합 (PR, Issue와 연동 쉬움)
- Marketplace에 수많은 사전 제작된 Action 존재
- Public 저장소는 무료로 무제한 사용 가능

GitLab CI

개요

- GitLab에 내장된 CI/CD 도구로, 저장소 루트의 .gitlab-ci.yml 파일 하나로 전체 파이프라인을 정의합니다.

핵심 개념

- **Pipeline:** 전체 CI/CD 프로세스 (GitHub Actions의 Workflow에 해당)
- **Stage:** 파이프라인 내 단계 (예: build → test → deploy). 같은 Stage 내 Job은 병렬 실행
- **Job:** Stage 안에서 실행되는 개별 작업

- **Runner:** 작업을 실제로 수행하는 에이전트

예시 YAML

stages:

- build
- test
- deploy

build-job:

stage: build

script:

- npm install
- npm run build

test-job:

stage: test

script:

- npm test

deploy-job:

stage: deploy

script:

- echo "배포 스크립트 실행"

only:

- main

장점

- Stage 개념이 명확해서 파이프라인 흐름을 이해하기 쉬움
- GitLab 자체에 Container Registry, 이슈 트래커 등이 통합되어 있어 올인원 환경 구성 가능
- Self-hosted GitLab 사용 시 완전한 온프레미스 환경 구축 가능 (보안 민감한 기업 환경에 유리)

두 도구 비교

항목	GitHub Actions	GitLab CI
설정 파일 위치	.github/workflows/*.yaml	.gitlab-ci.yml (루트)
파일 개수	여러 개 가능	하나로 통합
구조 단위	Workflow → Job → Step	Pipeline → Stage → Job
재사용 단위	Action (Marketplace)	Template, Include
On-premise	GitHub Enterprise Server 필요	GitLab Community/Enterprise Edition으로 쉽게 구축
보안 민감 환경	상대적으로 제약 있음	Self-hosted Runner로 완전 격리 가능

GD RDM 프로젝트 관점에서 참고하실 점

회사 보안 정책상 실제 데이터가 외부로 나갈 수 없는 환경이시라, 만약 CI/CD를 도입하신다면:

- **Self-hosted Runner**를 사내 서버에 설치하시면 GitHub Actions든 GitLab CI든 코드와 데이터가 외부로 나가지 않고 사내에서 자동 테스트/빌드가 가능합니다.
- 예를 들어 rdm_data_generator.py로 합성 데이터를 생성 → rdm_merge.py로 병합 → rdm_analysis.py로 분석하는 파이프라인 전체를 자동화된 테스트

트 흐름으로 구성하실 수 있습니다.

- 다만 사내 보안 정책상 외부 SaaS(GitHub.com, GitLab.com)의 Runner를 못 쓰실 가능성이 높으니, 이 부분은 사내 인프라팀과 먼저 확인이 필요하실 것 같습니다.
-

PR, Issue와 연동 쉬움 에서 PR과 Issue가 뭐야?

PR(Pull Request)과 Issue는 GitHub, GitLab 같은 협업 플랫폼에서 사용하는 핵심 기능입니다.

PR (Pull Request)

- 개발자가 작성한 코드 변경사항을 원본 저장소(main 브랜치 등)에 반영해 달라고 요청하는 기능입니다.
- 보통 이런 흐름으로 사용됩니다:
 1. 새 브랜치를 만들어 기능 개발
 2. 개발 완료 후 main 브랜치에 병합해 달라고 PR 생성
 3. 팀원들이 코드를 리뷰하고 댓글로 피드백
 4. 승인되면 병합(Merge)
- GitLab에서는 같은 개념을 ****MR(Merge Request)****이라고 부릅니다.

Issue

- 버그 리포트, 기능 요청, 할 일(Task) 등을 기록하고 추적하는 게시판 같은 기능입니다.
- 제목, 설명, 담당자, 라벨(bug/enhancement 등), 마감일을 지정할 수 있습니다.
- 예: "로그인 시 에러 발생" 같은 버그를 Issue로 등록하면, 해당 문제를 어느 커밋/PR에서 해결했는지 서로 연결해서 추적할 수 있습니다.

"PR, Issue와 연동 쉬움"의 의미

- GitHub Actions는 PR이 생성될 때 자동으로 테스트를 돌려서 결과를 PR 화면에 바로 표시해주거나, 특정 Issue 번호를 커밋 메시지에 적으면 자동으로 연결해주는 등 GitHub 자체 기능들과 매끄럽게 통합된다는 뜻입니다.